

Software Process Models

This chapter contains the following topics:

- **Waterfall Model**
- **Incremental Model**
- **Spiral Model**
- **V-Model**
- **Agile Models (Scrum, XP)**
- **RAD (Rapid Application Development)**
- **Prototyping Model**

1. Waterfall Model

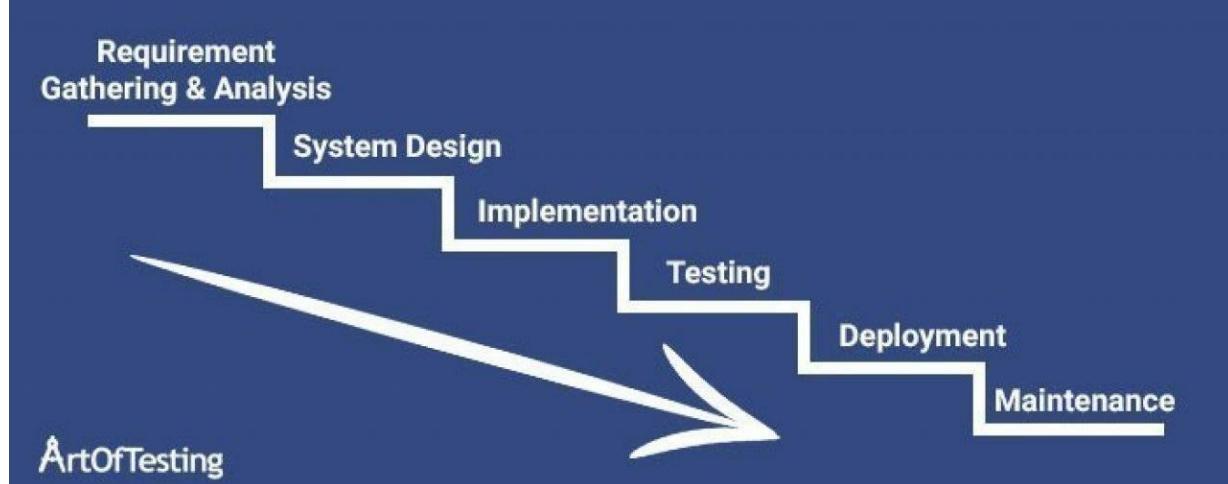
What is the Waterfall Model?

- The **Waterfall Model** is one of the **oldest and simplest** software development models. It works like a **linear staircase** each step must be **fully completed** before moving to the next one
- Once you go down a step, you **can't go back easily**, just like water flows down a waterfall that's why it's called the **Waterfall Model**.

Phases in Waterfall Model:

1. **Requirement Gathering:** Understand what the client wants.
Example: Client wants an app for managing school attendance.
2. **System Design:** Plan how the software will work.
Example: Decide how screens will look, where data will be stored, how users will log in, etc.
3. **Implementation (Coding):** Write the actual code based on the design.
Example: Developers start coding the login screen, attendance records, etc.
4. **Testing:** Test the software to find and fix errors.
Example: Check if the app works correctly when a teacher adds attendance.
5. **Deployment:** Deliver the finished software to the client.
Example: Launch the app on the client's devices or publish it online.
6. **Maintenance:** Fix issues or make small changes after release.
Example: A bug is found in the attendance summary, and it needs fixing.

Waterfall Model



When to Use the Waterfall Model:

Use it when:

- The requirements are **very clear and fixed** from the beginning.
- The project is **simple and short**.
- The client **won't change their mind** later.
- The team is **experienced** and the tools are stable.

Main Disadvantage

- Once a phase is completed, it's **very hard to go back** and make changes.
- If the client changes their mind or you find a mistake in earlier steps, you may need to **start over**.
- It doesn't handle **changing requirements** well.

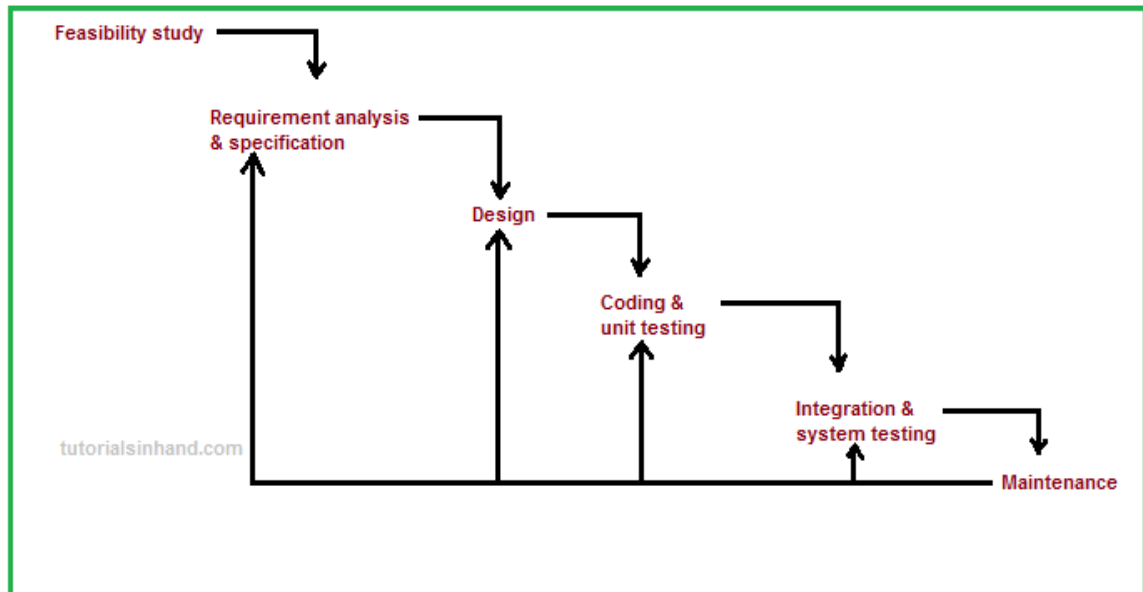
2.Iterative Waterfall Model

What is the Iterative Waterfall Model?

- The **Iterative Waterfall Model** is an **improved version** of the traditional Waterfall Model.
- In the **basic Waterfall Model**, once a phase is completed, you **can't go back** to make changes.
But in the **Iterative Waterfall Model**, you are **allowed to go back** to the previous phase if something goes wrong.

Main Idea:

- It still follows the same steps as Waterfall:
- **(Requirements → Design → Coding → Testing → Deployment),**
But now **feedback loops** are allowed — so, **you can revisit and fix** issues from earlier phases.



Iterative Waterfall Model

Advantages of Iterative Waterfall Model

- More **flexibility** than the original Waterfall
- **Errors can be corrected early**, saving cost and time
- Better **quality** because feedback is used
- Still follows a **structured and disciplined** approach

Disadvantages

- Not as flexible as Agile
- Still **requires full planning** before starting
- Frequent changes may lead to **more time and effort**
- Not suitable if **requirements keep changing a lot**

3.Incremental Model

What is the Incremental Model?

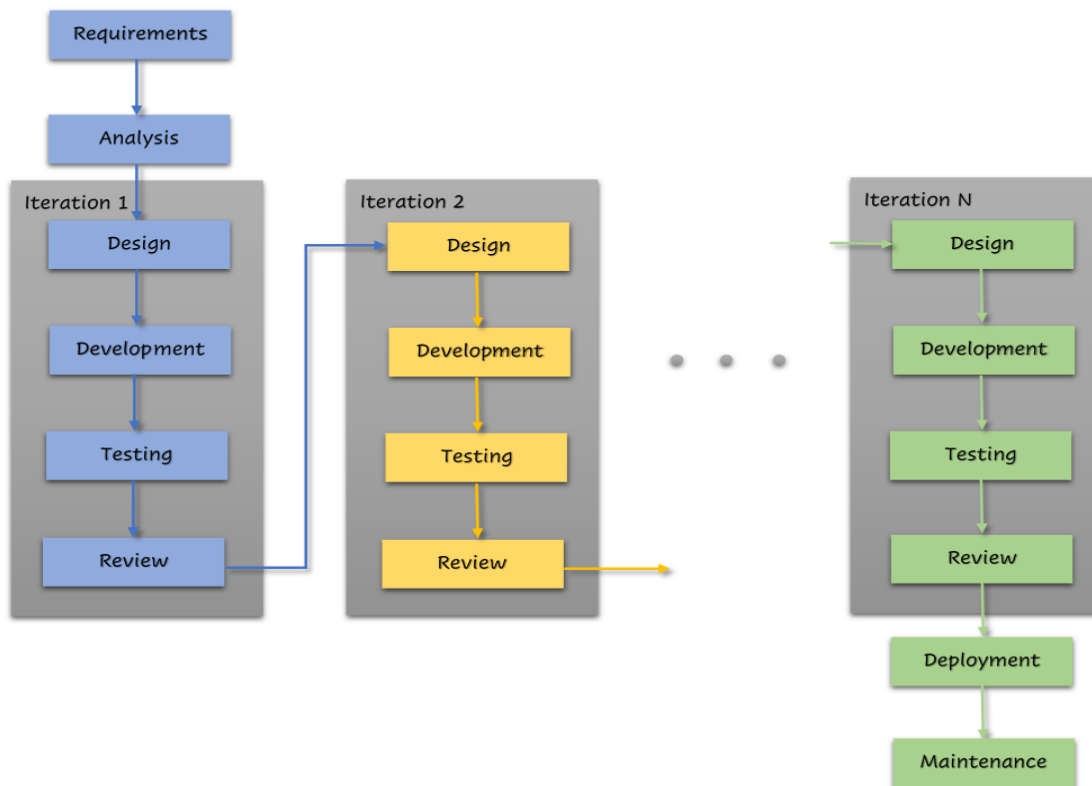
- The **Incremental Model** builds software **step-by-step in small parts** (called increments).
Each part adds **more features** to the previous version until the full software is ready.
- **Don't have to build everything at once** instead, develop and deliver the software in small pieces.

How It Works:

Lets say making an **online learning app**:

- **First Increment** – Build a basic version: user login and course list
- Deliver this to users, and let them use it.
- **Second Increment** – Add video playback feature
- Update the app and give it to users.
- **Third Increment** – Add quiz and certificate features
- Keep improving based on feedback.

Repeat this process until the software is complete.



Real-Life Example: Pizza Delivery

Think of ordering a **custom pizza**.

- First, you get the base and sauce.
- Then they add cheese.
- Then toppings like mushrooms or olives.
- Each step **adds more to your pizza**, and it gets better.

In the **Incremental Model**, each software increment adds more features just like pizza layers

When to Use the Incremental Model:

Use it when:

- You want to **deliver something early** to the client
- Requirements are **clear**, but you can build in parts
- You expect the client to **give feedback during development**
- The team wants to **divide work and finish faster**

Main Drawbacks:

- Planning all increments ahead of time can be hard.
- If the **basic version is weak**, future parts might not work well.
- Frequent integration (joining old + new parts) must be done carefully.

4.Spiral Model

What is the Spiral Model?

The **Spiral Model** is a software development model that combines the ideas of:

- **Design and development in steps (like Waterfall)**
- **Risk analysis and repeated improvement (like Iterative Model)**

It works in **cycles or “spirals”** each spiral includes:

- Planning
- Risk analysis
- Development
- Testing
- Feedback

Then the next spiral begins, improving the software each time.

Each spiral (round) has these 4 steps:

- Then repeat the cycle to add more features or fix problems.



- ### When to Use Spiral Model:

- @Hira-2415 Student at Zamindar College, Gujrat**

Drawbacks:

- **Costly and time-consuming** (suits large projects)
- Requires **experienced developers**
- Needs **careful planning and documentation**

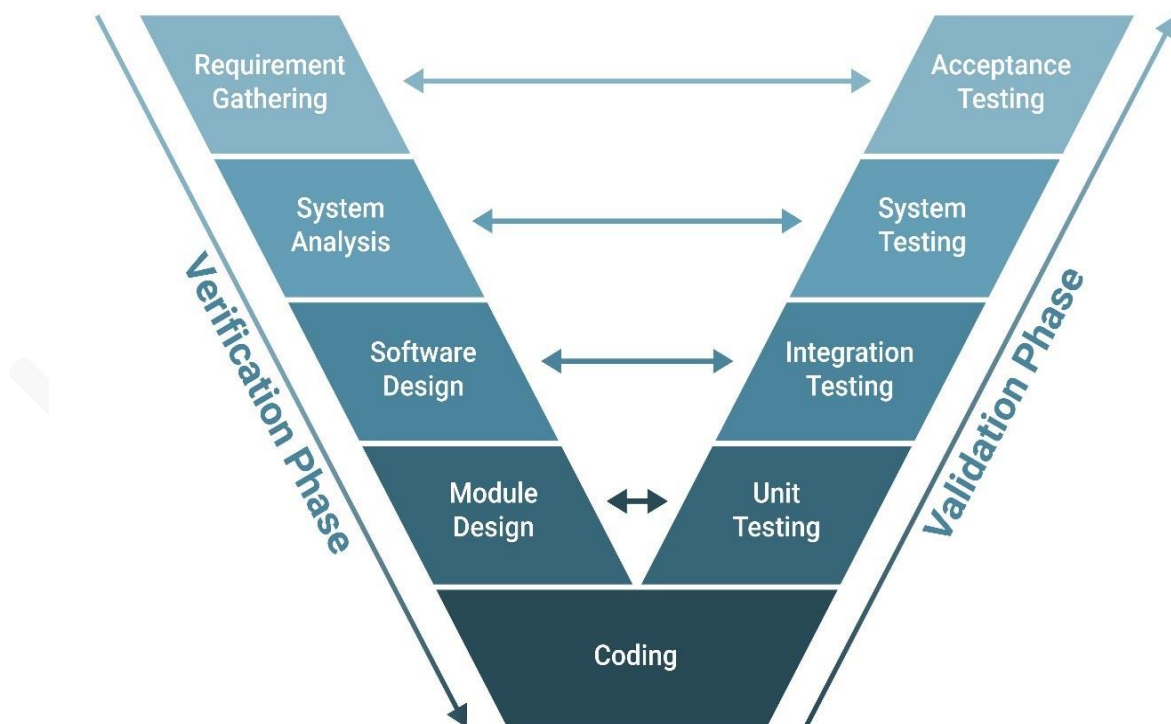
5. V-Model (Validation and Verification Model)

What is the V-Model?

- The **V-Model** is an extension of the Waterfall Model where **every development phase has a matching testing phase**. It looks like the letter “V” because of its structure.
- The left side of V = **Development**
- The right side of V = **Testing**
- The bottom = **Coding**

How It Works:

- For every phase on the left, there is a **related test** phase on the right.



Example: Making a Phone

1. Define what the phone should do (**requirements**) → Later you test if it meets those needs (**acceptance testing**)
2. Design how its parts will connect (**system design**) → Later you test all parts working together (**system testing**)
3. Design the screen and buttons (**component design**) → You test each unit like screen or button separately (**unit testing**)

When to Use V-Model

- When the **requirements are clear and fixed**
- In **safety-critical systems** (e.g., medical, aviation)
- When **strict testing** is needed

Drawbacks

- **Not flexible** for changes after development begins
- **Initial planning and documentation** take a lot of time
- Not good for **dynamic or fast-changing projects**

6. Prototyping Model

What is the Prototyping Model?

- The **Prototyping Model** is a software development approach where you **first build a quick, working version** of the software (called a **prototype**) to show to the client.
- This prototype is like a **rough draft**. It's not the final product, but it helps:
 - ✓ Show how the software might look or work
 - ✓ Get **feedback from the user**
 - ✓ Understand what the client **really needs**

Then, after the client approves or suggests changes, the **final version is developed** with proper coding and quality.

How It Works:

1. **Requirement Collection**
 - Try to understand what the client wants (even if it's not clear)
2. **Quick Design**
 - Make a basic layout or working model of the software (with limited features)
3. **Prototype Building**
 - Create a simple version that shows how the software will work
4. **User Review**
 - Show it to the client; they test it and give feedback
5. **Refining the Prototype**
 - Improve the prototype based on client suggestions

6. Final Development

- Now develop the **real software** with full features, based on the improved design

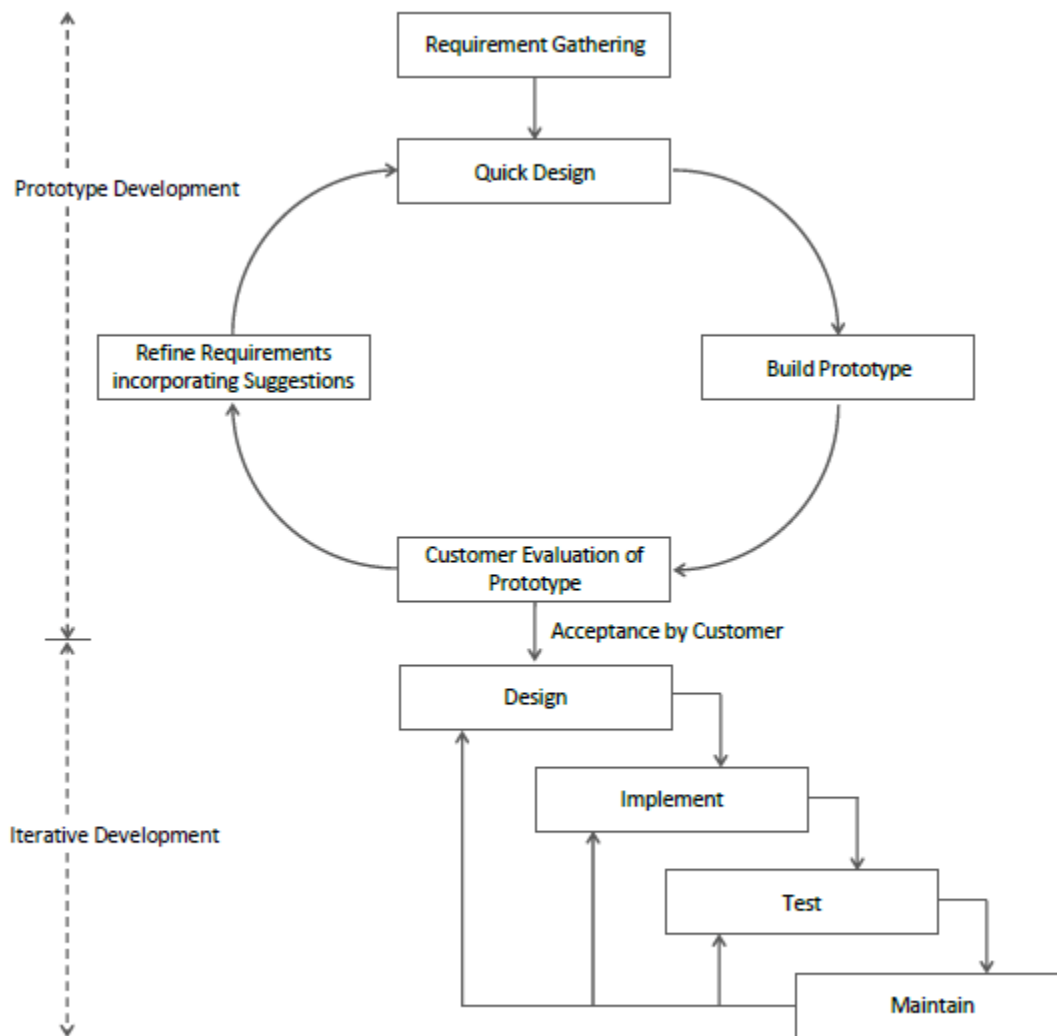


Figure 3: Prototype Model of Software Development

Real-Life Example: Website Sketch

Imagine you're a web designer.

- The client says: “I want a modern website.”
- But that’s not clear enough. So, you make a **quick sketch or demo site** to show:
 - Layout
 - Menu bar
 - Homepage design
- The client says: “Change the colors. Add a blog section.”

You take that feedback and now build the **real website** with the exact design the client wants. That's exactly how **prototyping** works in software.

When to Use the Prototyping Model

Use it when:

- The client is **not sure** what they want
- Requirements are **unclear or changing**
- You want to **avoid misunderstandings** between developers and users
- Fast feedback is important

Drawbacks of the Prototyping Model:

- Making many versions of the prototype can take **extra time**
- The client might **focus too much on the prototype design**, not the final product
- If the prototype is poor, it may give a **false idea** of the final product
- Sometimes the final development gets **delayed** due to too many changes

7.RAD – Rapid Application Development

What is the RAD Model?

RAD (Rapid Application Development) is a **quick and flexible** way to build software.

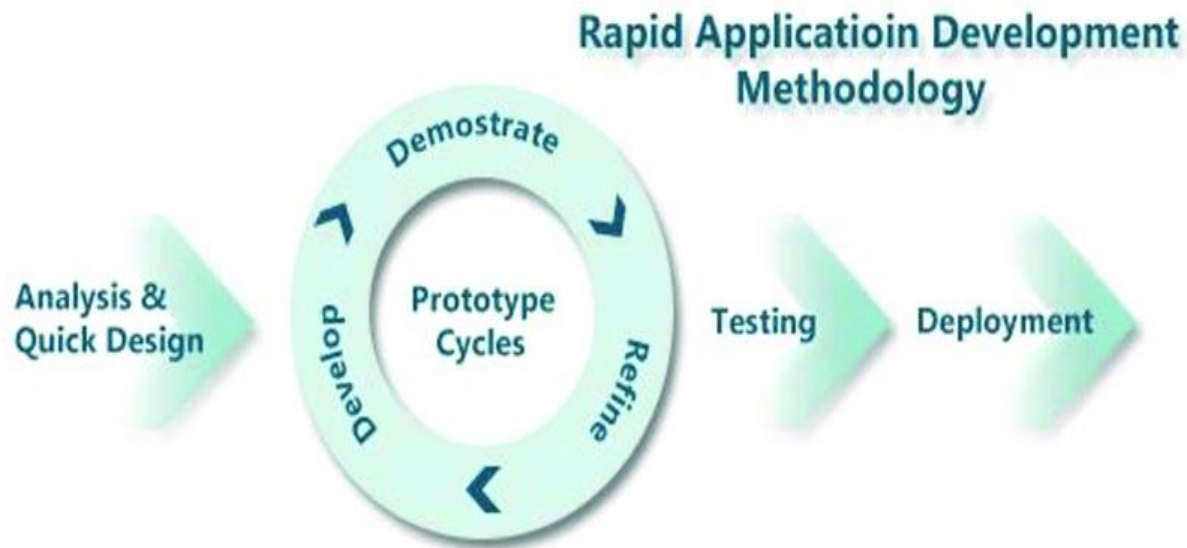
It focuses on **developing software fast** by:

- Using **prototypes** (rough versions)
- Reusing **pre-built components**
- Involving the **client at every step**
- Doing **less planning** and **more building**

The goal is to **deliver a working product quickly**, even if it's not perfect in the beginning.

How It Works:

1. **Requirements Planning**
 - Discuss with the client to understand the basic needs (keep it short).
2. **User Design**
 - Create a prototype (rough working model). Show it to the client.
 - Get **feedback**, make **quick improvements**, and repeat.
3. **Rapid Construction**
 - Start coding using reusable parts or ready-made modules.
 - Keep showing progress to the client.
4. **Testing**
 - Test each part as it is built (not at the end like in Waterfall).
 - Fix bugs fast based on user input.
5. **Deployment**
 - Deliver the final working version when the client is happy.



Real-Life Example: School Project with Templates

Let's say you have to make a project for school **quickly**.

- You use a **template** for the main layout
- You add your own content
- You show it to your teacher for suggestions
- You fix things and submit it — **fast and done!**

That's exactly how RAD works in software: **quick, flexible, and client-focused**.

When to Use RAD:

RAD works best when:

- You need to **deliver the software quickly**
- The **client is available** and gives regular feedback
- The project is **small or medium-sized**
- You can use **existing tools or modules** to save time

Drawbacks of RAD:

- **Not good for big or complex systems** (like banks, airlines)
- Needs a **team of experienced developers** to work fast
- Depends a lot on **client communication** — if the client is not available, RAD may fail
- May create a product that works fast but lacks **deep planning and structure**

8. Agile Models – Scrum & XP

What is Agile?

- **Agile** is a modern, flexible way of developing software.
- Instead of building the whole project at once (like in Waterfall), Agile breaks it into **small parts**, called **iterations** or **sprints**.
In each sprint, built
 - ✓ Build a **small feature**
 - ✓ **Test it**
 - ✓ **Get feedback**
 - ✓ Then **improve** it in the next sprint

This way, software gets better and better — with the help of **teamwork** and **customer involvement**.



Real-Life Example:

Imagine we are decorating a house.

- Decorate one room (1 sprint), then show it to the family.
- They give feedback: “Change the curtains.”
- We fix it, then move to the next room.

We are not waiting to finish the whole house. You are working **step-by-step** and improving along the way.

That's how **Agile works**.

Agile Key Features:

- **Short cycles** of work (called sprints)
- **Working software** is delivered regularly (every 1–2 weeks)
- **Customer is involved** from the start and gives feedback
- Focus on **teamwork**, communication, and flexibility
- Changes in requirements are **welcomed**, not feared

Agile Frameworks: Scrum & XP

(a):Scrum – The Most Popular Agile Framework:

- **Scrum** is a way to manage work in Agile, using a clear **team structure** and **sprint planning**.



Key Scrum Roles:

Role	Description
Product Owner	Talks to the client and decides what features are needed
Scrum Master	Makes sure the team follows Scrum rules and removes any obstacles
Development Team	Designers, developers, and testers who build the product

How Scrum Works:

- Work is divided into **sprints** (1–2 weeks)
- Before each sprint, the team decides **what to build**
- At the end of the sprint, the **team shows the result**
- The client gives **feedback**
- In the next sprint, the team **improves** and adds more features



Scrum Events (Meetings):

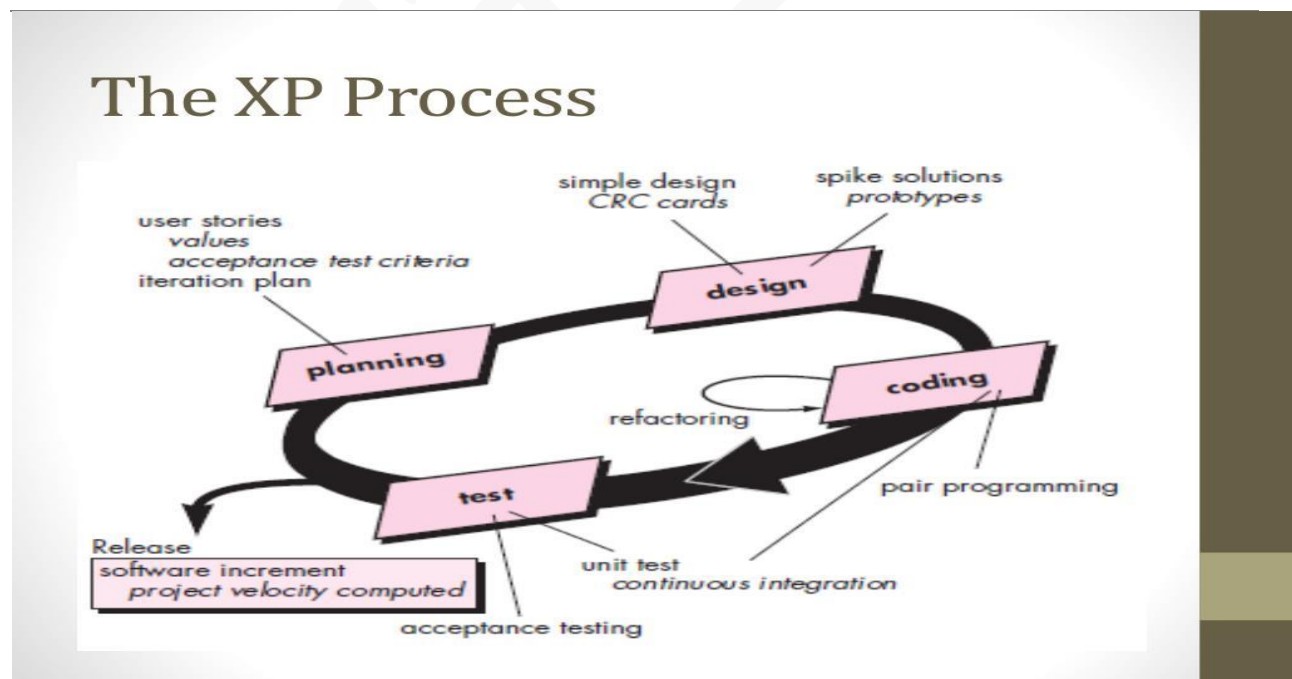
- **Sprint Planning** – Decide what to do this sprint
- **Daily Scrum (Stand-up)** – 15-minute daily team check-in
- **Sprint Review** – Show the completed work to client
- **Sprint Retrospective** – Team reviews what went well and what to improve

(b): XP (Extreme Programming)

- **XP (Extreme Programming)** is another Agile method, but it focuses mainly on **writing better code**.
- It encourages **frequent releases, good practices, and team collaboration**.

⚙️ XP Practices:

Practice	Explanation
Pair Programming	Two programmers work together at one computer — one writes code, the other reviews
Test-Driven Development (TDD)	Write tests before writing the code
Continuous Integration	Keep combining (merging) all code frequently to avoid conflicts
Refactoring	Keep improving the code without changing how it works
Simple Design	Keep the design simple and clean



Agile is Best For:

- Projects with **changing requirements**
- **Startups**, where speed and flexibility are important
- Teams that can **communicate frequently**
- Clients who want to see **progress quickly**

Drawbacks of Agile:

- Needs a **skilled and experienced team**
 - Requires **daily communication**
 - Not suitable if the client is **not available** for feedback
 - Hard to apply in **very large organizations** without training
-